

Membangun Language Model Kecil dari Nol

Dari Bigram ke GPT Subword

Dokumen ini merangkum eksperimen pertama membangun language model.

Fokusnya bukan mengejar model besar.

Fokusnya memahami alur lengkap:

```
dataset -> tokenizer -> batch -> model -> loss -> gradient -> generation
```

Kode utama yang dibahas:

- `bigram.py`
- `gpt.py`
- `gpt_subword.py`
- `load_alpaca_dataset.py`

Basic yang diasumsikan:

- Pernah memakai CNN, LSTM, ViT, atau BERT.
- Memahami tensor dan training loop secara umum.
- Tidak memerlukan matematika berat untuk mengikuti penjelasan.

1. Tujuan Language Model

Language model belajar memprediksi token berikutnya.

Contoh:

```
Input  : Saya sedang belajar  
Target : AI
```

Model tidak langsung menulis satu paragraf.

Model hanya melakukan satu langkah:

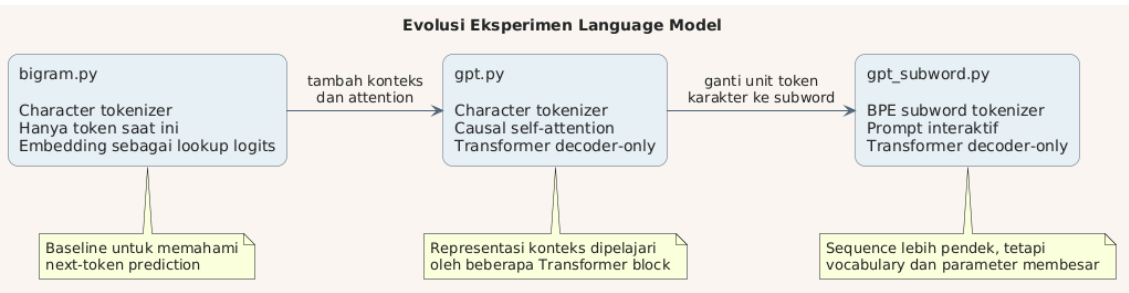
$P(\text{token berikutnya} \mid \text{token-token sebelumnya})$

Setelah satu token dipilih, token itu ditambahkan ke input.
Proses kemudian diulang.

Saya sedang belajar
Saya sedang belajar AI
Saya sedang belajar AI dari
Saya sedang belajar AI dari nol

Inilah arti **autoregressive generation**.

2. Evolusi Eksperimen



Ada tiga tahap model.

File	Tokenizer	Konteks	Model
bigram.py	Karakter	Praktis satu token	Lookup table
gpt.py	Karakter	Banyak token sebelumnya	Transformer
gpt_subword.py	BPE subword	Banyak subword sebelumnya	Transformer

Perubahan pertama adalah kemampuan melihat konteks.

Perubahan kedua adalah unit token.

Arsitektur `gpt.py` dan `gpt_subword.py` hampir sama.

Yang paling berbeda adalah tokenizer dan ukuran vocabulary.

3. Dataset Menjadi Token

Model neural network tidak menerima string.

Model menerima integer.

```
"Saya belajar"  
  ↓ tokenizer  
[50, 12874, 1689]
```

Integer tersebut adalah ID token.

ID token kemudian masuk ke embedding table.

Tokenizer bukan Transformer encoder

Di `gpt_subword.py` terdapat:

```
enc.encode(text)
```

Nama `encode` kadang membingungkan.

Ini hanya tokenisasi.

Ia bukan encoder seperti encoder pada BERT.

Ia tidak memahami makna kalimat.

Pemahaman dipelajari oleh parameter Transformer.

4. `bigram.py` : Baseline Paling Sederhana

Bigram model melihat token saat ini.

Lalu memprediksi token berikutnya.

```
token sekarang -> probabilitas token berikutnya
```

Bagian intinya:

```
self.token_embedding_table = nn.Embedding(vocab_size, vocab_size)
```

Biasanya embedding menghasilkan representasi tersembunyi.

Di model bigram ini, setiap baris langsung menjadi logits.

Jika vocabulary berisi 65 karakter:

```
input token ID
    ↓
lookup satu baris berisi 65 skor
    ↓
probabilitas 65 kandidat karakter berikutnya
```

Keterbatasannya

Bigram tidak benar-benar menggunakan sejarah panjang.

Prediksi berikut ini dianggap sama:

```
"Saya makan nasi"
"Kucing melihat nasi"
```

Jika token terakhir sama-sama **i**, bigram hanya melihat **i**.

Konteks sebelumnya hilang.

Karena itu hasilnya cepat belajar bentuk lokal.

Tetapi sulit menjaga kalimat panjang tetap konsisten.

5. Input dan Target yang Digeser

Training data dibuat dengan menggeser target satu token.

```
tokens = [Saya, sedang, belajar, AI]
```

```
x = [Saya, sedang, belajar]
```

```
y = [sedang, belajar, AI]
```

Pada setiap posisi:

```
Saya          -> sedang
```

```
Saya sedang   -> belajar
```

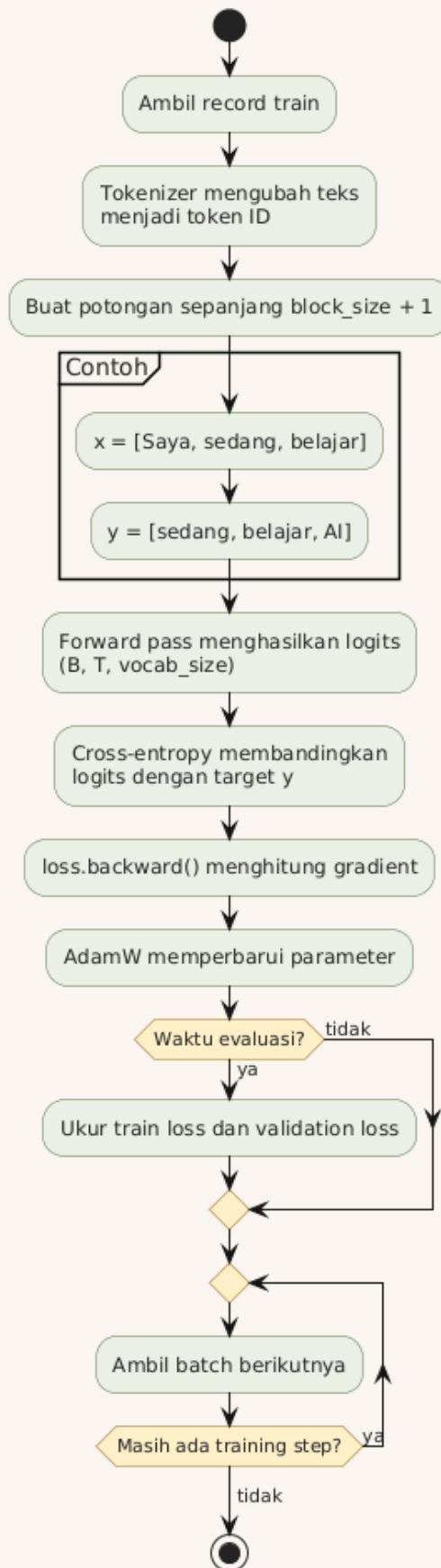
```
Saya sedang belajar -> AI
```

Satu sequence memberikan beberapa contoh training sekaligus.

`block_size` menentukan panjang maksimum sequence itu.

`batch_size` menentukan berapa sequence independen diproses bersama.

Alur Next-Token Training



6. Bentuk Tensor Utama

Notasi yang sering muncul:

```
B = batch size
T = time atau sequence length
C = channel atau embedding dimension
V = vocabulary size
```

Input token:

```
idx.shape = (B, T)
```

Setelah embedding:

```
x.shape = (B, T, C)
```

Setelah LM head:

```
logits.shape = (B, T, V)
```

Ini mirip cara ViT mengubah patch menjadi embedding.

Bedanya:

```
ViT : patch gambar -> embedding
GPT : token teks   -> embedding
```

Keduanya lalu memproses sequence embedding dengan Transformer.

7. `gpt.py` : Character-Level Transformer

`gpt.py` masih menggunakan karakter sebagai token.

Tetapi modelnya bukan bigram lagi.

Ia sudah memiliki:

- Token embedding.
- Position embedding.
- Multi-head self-attention.
- Causal mask.
- Feed-forward network.
- Residual connection.
- LayerNorm.
- LM head.



Token embedding

```
self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
```

Token ID diubah menjadi vektor berukuran `n_embd`.

Vektor ini dipelajari selama training.

Karakter `a` tidak memiliki embedding bermakna di awal.

Maknanya terbentuk melalui gradient update.

Position embedding

```
self.position_embedding_table = nn.Embedding(block_size, n_embd)
```

Self-attention sendiri tidak mengetahui urutan.

Position embedding memberi informasi posisi token.

```
token embedding + position embedding
```

Analogi sederhananya:

```
apa tokennya + token itu berada di mana
```

8. Self-Attention

Self-attention membuat satu token membaca token lain.

Setiap token menghasilkan tiga representasi:

```
Query : apa yang sedang saya cari?  
Key   : informasi apa yang saya miliki?  
Value : isi apa yang akan saya berikan?
```

Bagian utamanya:

```
wei = q @ k.transpose(-2, -1) * k.shape[-1] ** -0.5
```

`q @ k.T` menghasilkan skor hubungan antarposisi.

Bentuknya:

```
(B, T, head_size) @ (B, head_size, T)
      ↓
      (B, T, T)
```

Setiap token mendapatkan skor terhadap semua token lain.

Scaling

```
* head_size ** -0.5
```

Scaling menjaga skor tidak terlalu besar.

Tanpa scaling, softmax bisa terlalu tajam.

Satu token bisa mendapatkan probabilitas hampir satu terlalu cepat.

Causal mask

```
wei = wei.masked_fill(tril == 0, float('-inf'))
```

Token tidak boleh melihat masa depan.

Saat memprediksi token ketiga:

```
boleh melihat : token pertama dan kedua
tidak boleh   : token keempat dan seterusnya
```

Nilai `-inf` menjadi probabilitas nol setelah softmax.

9. Multi-Head Attention

Satu attention head mempelajari satu ruang hubungan.

Beberapa head berjalan paralel.

```
torch.cat([h(x) for h in self.heads], dim=-1)
```

Secara intuitif, head berbeda dapat fokus pada pola berbeda.

Contohnya:

- Hubungan subjek dan kata kerja.
- Token yang mengawali paragraf.
- Nama entitas yang disebut sebelumnya.
- Pola tanda baca.

Interpretasi head tidak selalu sesederhana itu.

Namun analogi tersebut cukup membantu.

Syarat implementasi saat ini:

```
n_embd harus habis dibagi n_head
```

Dengan:

```
n_embd = 128  
n_head = 4
```

Maka ukuran setiap head adalah 32.

10. Feed-Forward dan Residual

Attention adalah tahap komunikasi antar-token.

Feed-forward adalah tahap komputasi per-token.

```
Linear(n_embd, 4 * n_embd)
ReLU
Linear(4 * n_embd, n_embd)
```

Setiap posisi melewati MLP yang sama.

Residual connection mempertahankan jalur informasi:

```
x = x + attention(...)
x = x + feed_forward(...)
```

Konsep ini sama seperti skip connection pada ResNet.

LayerNorm membantu menjaga aktivasi tetap stabil.

11. Decoder-Only vs BERT

Basic BERT sangat membantu untuk memahami perbedaannya.

BERT	GPT
Encoder-only	Decoder-only
Melihat kiri dan kanan	Hanya melihat token sebelumnya
Masked language modeling	Next-token prediction
Cocok untuk representasi/klasifikasi	Cocok untuk generation

Istilah decoder-only berasal dari Transformer original.

GPT tidak memiliki encoder dan cross-attention.

Ia hanya menggunakan causal self-attention.

12. Loss dan Backpropagation

LM head menghasilkan logits.

Logits adalah skor mentah.

```
loss = F.cross_entropy(logits, targets)
```

Cross-entropy memberi penalti ketika target mendapat probabilitas kecil.

Saat training:

```
optimizer.zero_grad(set_to_none=True)  
loss.backward()  
optimizer.step()
```

Urutannya:

```
hapus gradient lama  
-> hitung gradient baru  
-> perbarui parameter
```

Parameter yang diperbarui mencakup:

- Embedding.
- Proyeksi Q, K, dan V.
- Attention output projection.
- Feed-forward layers.
- LayerNorm.
- LM head.

Jadi bukan hanya "decoder bagian akhir" yang dilatih.

Seluruh model dilatih end-to-end.

13. Generation

Saat generation, model hanya memakai logits posisi terakhir.

```
logits = logits[:, -1, :]  
probs = F.softmax(logits, dim=-1)  
idx_next = torch.multinomial(probs, num_samples=1)
```

Langkahnya:

```
logits  
-> softmax  
-> probabilitas token  
-> sampling satu token  
-> append ke context  
-> ulangi
```

Model tidak mengambil kalimat utuh dari database.

Ia membentuk teks token demi token.

Kualitas generation dipengaruhi oleh:

- Kualitas model.
- Dataset.
- Context prompt.
- Temperature.
- Top-k atau top-p.
- Panjang konteks.

Implementasi saat ini memakai sampling langsung.

Temperature dan top-k/top-p belum ditambahkan.

14. `gpt_subword.py` : Dari Karakter ke BPE

Character tokenizer memiliki vocabulary kecil.

Tetapi sequence menjadi panjang.

```
"belajar" -> b, e, l, a, j, a, r
```

Subword tokenizer dapat menghasilkan potongan yang lebih besar.

```
"belajar" -> satu atau beberapa subword
```

Keuntungannya:

- Sequence lebih pendek.
- Pola kata dapat dipelajari lebih langsung.
- Lebih cocok untuk model modern multilingual.

Trade-off-nya:

- Vocabulary lebih besar.
- Token embedding lebih besar.
- LM head lebih besar.

Mengapa parameter langsung naik?

Ukuran token embedding:

```
vocab_size x n_embd
```

Ukuran LM head hampir sama:

```
n_embd x vocab_size
```

Dengan vocabulary sekitar 50 ribu dan `n_embd = 128`:

```
embedding sekitar 6,4 juta parameter  
LM head    sekitar 6,4 juta parameter
```

Dua matriks tersebut sudah sekitar 12,8 juta parameter.

Karena itu model subword tercatat sekitar 13,7 juta parameter.

Catatan encoding saat ini

Kode aktif saat ini memakai:


```
enc = tiktoken.get_encoding('gpt2')
```

`cl100k_base` masih tersedia sebagai komentar.

`cl100k_base` lebih modern dan multilingual.

Namun vocabulary-nya sekitar 100 ribu.

Parameter embedding dan LM head akan lebih besar lagi.

Untuk Gatra kecil, tokenizer BPE Indonesia sendiri lebih masuk akal.

15. Dataset JSONL

Dataset awal berupa satu file teks panjang.

Eksperimen berikutnya memakai JSON Lines:

```
{"text": "Saya sedang belajar neural network."}  
{"text": "Model ini melakukan next-token prediction."}
```

Satu baris adalah satu objek JSON.

Keuntungannya:

- Batas record tetap jelas.
- Bisa menyimpan metadata.
- Mudah dibaca secara streaming.
- Mudah dideduplikasi.
- Mudah dipisah per sumber atau domain.

JSONL bukan bentuk akhir yang masuk ke GPU.

Trainer tetap mengambil field `text`.

Teks kemudian ditokenisasi menjadi sequence integer.

16. Shuffle dan Train/Validation Split

Record diacak dengan seed tetap:

```
random.Random(1337).shuffle(records)
```

Yang diacak adalah urutan record.

Bukan karakter.

Bukan token di dalam kalimat.

Jika token diacak, struktur bahasa akan rusak.

Split dilakukan pada level record:

```
record_split = int(0.9 * len(records))
```

Ini lebih aman daripada menggabungkan semua teks lalu memotong di tengah.

Satu record hanya berada di train atau validation.

Tujuannya mengurangi leakage.

17. `load_alpaca_dataset.py`

Script ini mengambil 20 sampel dari Hugging Face:

```
load_dataset(  
    "AnnasBlackHat/alpaca-indonesia-llama",  
    split="train",  
)
```

Dataset sumber memiliki struktur agak unik.

Field `text` berisi JSON string lagi.

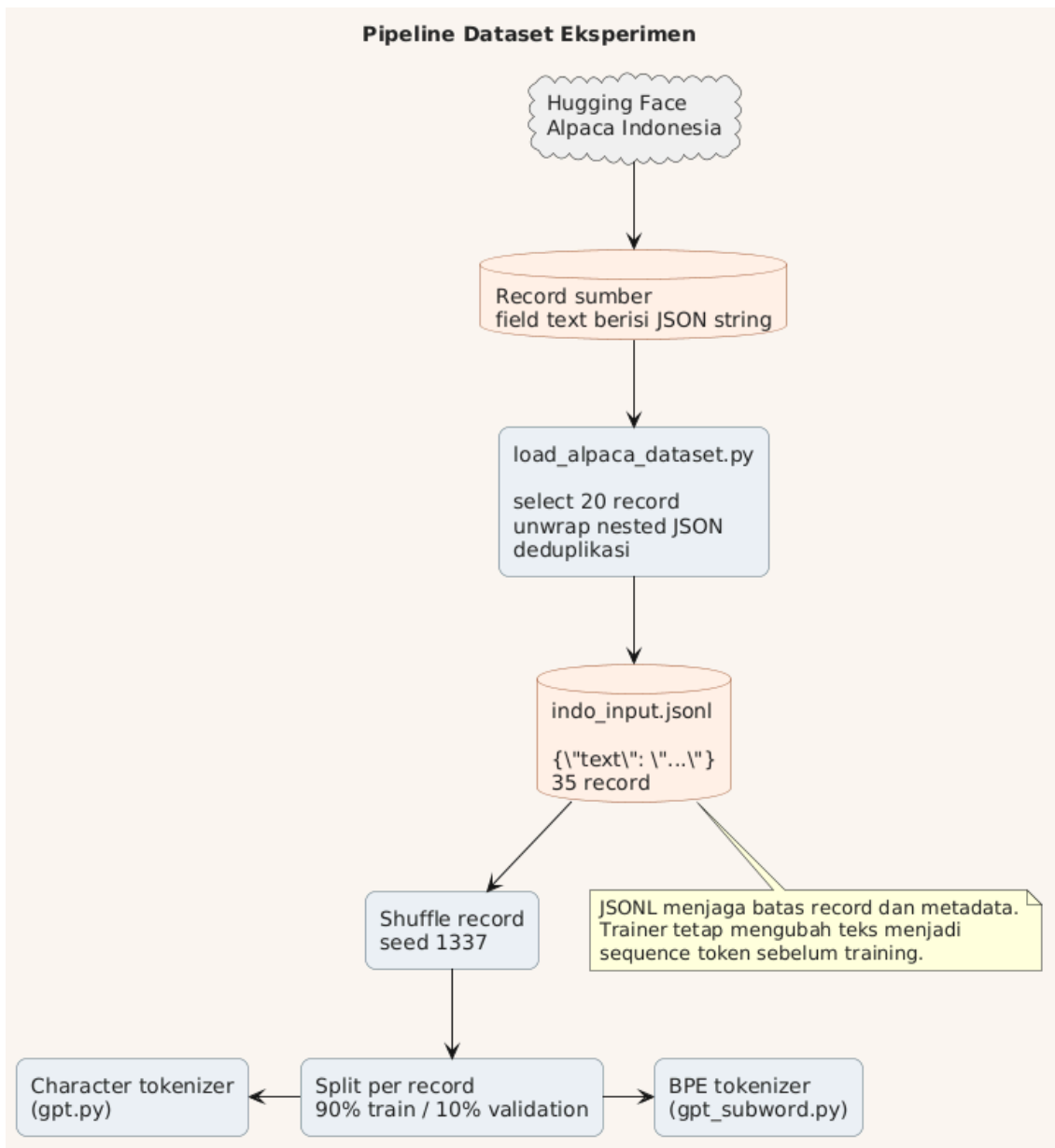
Karena itu perlu di-unwrap:

```
json.loads(row["text"])["text"]
```

Script kemudian:

1. Memilih 20 record.

2. Mengubahnya menjadi `{"text": ...}`.
3. Membaca data yang sudah ada.
4. Menghapus calon duplikasi.
5. Menambahkan record baru ke `indo_input.jsonl`.



Idempotent sederhana

Script menyimpan semua teks lama dalam sebuah set:

```
seen = {record["text"] for record in existing}
```

Record yang sudah ada tidak ditambahkan lagi.

Jadi script dapat dijalankan ulang tanpa terus menggandakan 20 record yang sama.

18. Alpaca dan Instruction Tuning

Record Alpaca memiliki pola:

```
### Human: pertanyaan
### Assistant: jawaban
```

Alpaca dibuat untuk instruction tuning.

Ia bukan dataset base pretraining murni.

`### Human:` dan `### Assistant:` saat ini hanya teks biasa.

Keduanya bukan special token.

Tokenizer akan memecahnya menjadi beberapa token.

Menuju format chat yang lebih baik

Format penyimpanan modern biasanya memakai `messages` :

```
{
  "messages": [
    {"role": "user", "content": "Apa ibu kota Indonesia?"},
    {"role": "assistant", "content": "Ibu kota Indonesia adalah Jakarta."}
  ]
}
```

Chat template kemudian mengubahnya menjadi sequence.

Contohnya:

```
<|im_start|>user
Apa ibu kota Indonesia?<|im_end|>
<|im_start|>assistant
Ibu kota Indonesia adalah Jakarta.<|im_end|>
```

Special token harus didaftarkan di tokenizer.

Menulis string yang terlihat khusus belum membuatnya menjadi satu token khusus.

19. Base Pretraining vs SFT

Saat ini semua record diratakan menjadi teks.

Loss dihitung pada semua token.

Ini cukup untuk eksperimen pipeline.

Namun tahap serius sebaiknya dipisahkan.

Base pretraining

Dataset:

```
{"text": "Artikel, buku, cerita, kode, atau dokumen."}
```

Loss dihitung pada semua token.

Tujuannya belajar bahasa dan pengetahuan umum.

Supervised fine-tuning

Dataset:

```
{"messages": [...]}
```

Token system dan user biasanya tidak dihitung dalam loss.

Labelnya diubah menjadi **-100**.

Token assistant tetap dihitung.

```
user prompt      -> label -100
assistant answer -> label token asli
```

Tujuannya mengajarkan model cara menjawab.

20. Membaca Train dan Validation Loss

Eksperimen subword menghasilkan pola seperti:

```
train loss turun terus  
validation loss turun lalu stagnan/naik
```

Ini menunjukkan overfitting.

Model semakin bagus mengingat train set.

Tetapi kemampuan pada record validation tidak ikut membaik.

Dataset hanya puluhan record.

Model memiliki jutaan parameter.

Rasionya sangat tidak seimbang.

Karena itu output mencampur fragmen:

- Julius Caesar.
- Atom.
- Commodore 64.
- Format `### Human` .
- Potongan kalimat buatan sendiri.

Pipeline bekerja.

Model hanya belum memiliki data cukup untuk generalisasi.

21. CUDA dan Python

Kode model ditulis dalam Python.

Namun operasi berat tidak dihitung oleh interpreter Python.

```
Python/PyTorch  
-> C++ runtime  
-> CUDA/cuBLAS kernels  
-> RTX 4060
```

Contoh:

```
q @ k.transpose(-2, -1)
```

Python mendeskripsikan operasi.

GPU menjalankan matrix multiplication.

Ini alasan Python tetap umum untuk riset model.

Riset cepat ditulis.

Komputasi berat tetap berjalan di native code.

22. Apa yang Sudah Berhasil?

- Environment dikelola dengan `uv`.
- PyTorch CUDA terdeteksi pada RTX 4060.
- Bigram model berhasil dilatih.
- Character-level GPT berhasil dilatih.
- Subword GPT berhasil dilatih.
- Prompt interaktif berhasil dipakai saat generation.
- Dataset berpindah dari TXT ke JSONL.
- Record di-shuffle dan di-split secara reproducible.
- Sampel Alpaca Indonesia berhasil dimuat.
- Train dan validation loss dapat dipantau.
- Overfitting pada dataset kecil berhasil diamati secara nyata.

Ini sudah merupakan pipeline language model end-to-end.

Masih kecil.

Tetapi setiap komponen intinya sudah terlihat.

23. Keterbatasan Implementasi Saat Ini

- Dataset sangat kecil.
- Data pretraining dan instruction masih tercampur.

- Tokenizer belum dilatih khusus Indonesia.
- Belum ada checkpoint dan resume.
- Belum ada mixed precision.
- Belum ada learning-rate scheduler.
- Belum ada temperature, top-k, atau top-p.
- Belum ada assistant-only loss masking.
- Belum ada weight tying.
- Position embedding masih learned absolute embedding.
- Feed-forward masih memakai ReLU.
- Attention belum memakai optimized SDPA/Flash Attention API.
- Evaluasi baru berfokus pada loss dan contoh output.

Keterbatasan ini bukan kegagalan.

Ini daftar eksperimen berikutnya.

24. Arah Gatra

Gatra diposisikan sebagai general-purpose language model.

Arah yang masuk akal:

Indonesia-first, bilingual small language model untuk reasoning dan tool use.

Dua track dapat berjalan bersama.

Track riset

```
Gatra-1M  
-> Gatra-10M  
-> Gatra-30M  
-> Gatra-100M
```

Model dilatih dari nol.

Tujuannya memahami tokenizer, arsitektur, data, scaling, dan training dynamics.

Track produk

```
pretrained model 0.5B-3B
-> continued pretraining Indonesia
-> SFT
-> preference optimization
-> tools/RAG
-> quantization
```

Jalur ini lebih cepat menghasilkan model yang berguna.

25. Langkah Berikutnya

Urutan yang paling masuk akal:

1. Pisahkan `pretrain.jsonl` dan `sft.jsonl`.
2. Latih tokenizer BPE Indonesia 8k atau 16k.
3. Tambahkan checkpoint dan resume.
4. Tambahkan batas waktu mode `smoke`, `study`, dan `run`.
5. Tambahkan metrics token per detik dan penggunaan VRAM.
6. Tambahkan temperature, top-k, dan top-p.
7. Tambahkan weight tying.
8. Modernisasi ke RoPE, RMSNorm, dan SwiGLU.
9. Buat Gatra-10M sebagai baseline.
10. Bandingkan Gatra-30M pada token budget yang sama.

Target dekatnya bukan model frontier.

Target dekatnya adalah eksperimen yang dapat dijelaskan.

Setiap perubahan harus menjawab satu pertanyaan.

Setiap hasil harus dapat dibandingkan dengan baseline.

Penutup

Bigram mengajarkan inti probabilitas token berikutnya.

GPT karakter menambahkan konteks dan self-attention.

GPT subword mengganti unit bahasa menjadi potongan yang lebih efisien.

JSONL dan loader Alpaca mulai memperkenalkan data engineering.

Train/validation loss memperkenalkan evaluasi dan overfitting.

Semua model tersebut tetap memiliki inti yang sama:

```
lihat token sebelumnya  
-> hitung logits  
-> prediksi token berikutnya  
-> belajar dari kesalahan  
-> ulangi
```

Skala dapat berubah dari ribuan menjadi miliaran parameter.

Prinsip dasarnya tetap dapat ditelusuri dari kode kecil ini.